OLO ACADEMY · AIEP CAPSTONE · TRACK A

3-Eye RoadScan Technical Report

From a phone photo of a road to a deduplicated, prioritised repair queue

Team	Sarah Al-Bahesh · Rimas Al-Abunasser · Fajer Alshahrani
Track	A: RoadScan, Road Damage Intelligence for Municipalities
Repository	github.com/saravii7/3-EyeRoadScanProject
Live application	3eye-roadscan.streamlit.app
Adopted detector	YOLOv8s run 17 · confidence 0.46 · 640 px · SHA-256 25b412ff...d15f
Date	November 2026

0.959
mAP@0.5, China development set (4 classes)

0.764
mAP@0.5, US held-out country

0.262
mAP@0.5, our 190 local photos

8 / 9
duplicates caught on test pairs; 0 / 24 false merges on test, 4 / 36 on tuning (GPS on)

Contents

1. Problem, user and success criteria
2. Data card
3. System architecture
4. Baseline and main approach
5. Experiments E1 to E5
6. Evaluation on held-out and team-built sets
7. Error analysis
8. Application, latency and artifact
9. Responsible AI
10. Conclusion and next steps
11. Contribution statement

References

Appendix A. Reproducibility
Appendix B. Experiment log (runs 3 to 28)
Appendix C. Full country and class tables
Appendix D. Track A checklist mapping
Appendix E. Concepts for Q&A
Appendix F. Problems we hit and how we fixed them

Every number in this report comes from a saved result file in the repository (paths given in Appendix A) or from a measurement described in the text. Development, held-out and team-built sets are reported separately and are never pooled.

1. Problem, user and success criteria

Problem. Municipalities receive many road-damage photos from citizens and inspection vehicles. Today an inspector looks at each photo by hand, the same pothole is often reported several times, and there is no consistent rule for what to repair first. The result is slow triage, wasted crew visits and inconsistent decisions.

Users and the decision we support.

User	Need	What RoadScan gives them
Maintenance supervisor	A ranked list of what to fix first; avoid sending two crews to the same pothole.	Queue sorted by a 0 to 100 priority score; duplicate warnings with similarity and distance.
Inspection vehicle operator	Fast, reliable detection on many images.	Batch upload, ~0.2 s detection per image on CPU, GeoJSON export.
Citizen	Their photo is understood and not wrongly ignored as a duplicate.	Boxes, damage type and priority shown back; strict duplicate thresholds so real reports are not hidden.

The central question (from the brief): *will a model trained on roads in other countries work on roads in our city?* We answer it directly with a held-out country (USA) and a team-collected local set from our own city (Section 6).

Success criteria. We define "good enough" per component in terms of the user's cost of errors:

Criterion	Why it matters to the user	Target	Result
Pothole recall on development data at the chosen threshold	A missed pothole is a safety risk; a false alarm only costs a review.	≥ 90 %	**96.6 % ✔**
Detector beats the image-level baseline	Localisation must add value to justify its complexity.	Clear margin, same images	**0.962 vs 0.496 macro-F1 ✔**
False duplicate merges with the chosen setting (same class + within 30 m + cosine ≥ 0.80)	A false merge hides a real report.	0 with GPS	**0 / 24 on 9 test pairs ✔** **4 / 36 on 10 tuning pairs**
Ranking stable under ±20 % weight changes	The queue should not depend on an arbitrary weight.	Top-10 unchanged	**10 / 10 in all 8 tests ✔**
CPU time per image (detection)	Bulk processing by municipalities.	< 1 s	**0.16 to 0.19 s ✔**
Detection recall on our local roads	The real deployment condition.	Comparable to held-out country	**17 % vs 71 % ✘**

The last row is the honest headline: the system works end to end, but the detector is **not yet ready for deployment on our local roads**. Sections 5 to 7 quantify why and Section 10 gives the plan to close the gap.

2. Data card

Dataset	Source and licence	Size	Fields	Role
RDD2022	Arya et al., CRDDC 2022, figshare release; CC BY 4.0	~47,000 images from 6 countries; Pascal VOC XML boxes	image, box (xmin, ymin, xmax, ymax), class name, country	Training, development, held-out test
Local test set (team-built)	Our own phone photos of roads in our city, taken on foot; labelled by the team in YOLO format (AI-assisted pre-labels, every box reviewed by the team)	190 images: 134 with damage (333 boxes), 56 healthy roads	image, YOLO box, class, EXIF GPS	Test only, never trained on
Duplicate pairs (team-built)	Our own photos: the same damage photographed twice from different angles, plus different damages	60 photos = 30 pairs; 19 usable (10 tuning, 9 test)	image, pair id, EXIF GPS (59 of 60 have GPS)	Threshold tuning and E5

Preparation (owned by Sarah)

1. **Conversion.** RDD2022 XML was converted to YOLO format (`scripts/data/convert_to_yolo.py`). Only D00, D10, D20 and D40 are kept; other RDD classes (e.g. repairs, white-line blur) are dropped and counted. Images without any kept box stay in the set as negatives.
2. **Country split, not random.** `split_by_country.py` separates countries. Consecutive frames from the same drive are nearly identical, so a random split would leak near-duplicates into the test set and inflate mAP. Splitting by country removes this route-level leakage between training and test.
3. **Sampling.** v1: 600 images (200 per country) for a quick baseline. v2: **8,999 training images** (Japan 4,000 · India 3,500 · Czech 1,499), fixed seed 42, enriched for the rarer classes D10 and D40 (`make_sample_v2.py`). Image lists are versioned in `data/manifests/`.

Split	Countries	Images	Labelled boxes	Used for
Train (v2)	Japan, India, Czech	8,999	1,427 images empty	Fine-tuning detector and baseline
Development	China_MotorBike	1,977	4,650	Model choice, confidence threshold, E1 to E3
Held-out country	United States	4,804	11,010	E4, measured once with frozen settings
Local test	Our city	190	333	E4 and deployment question

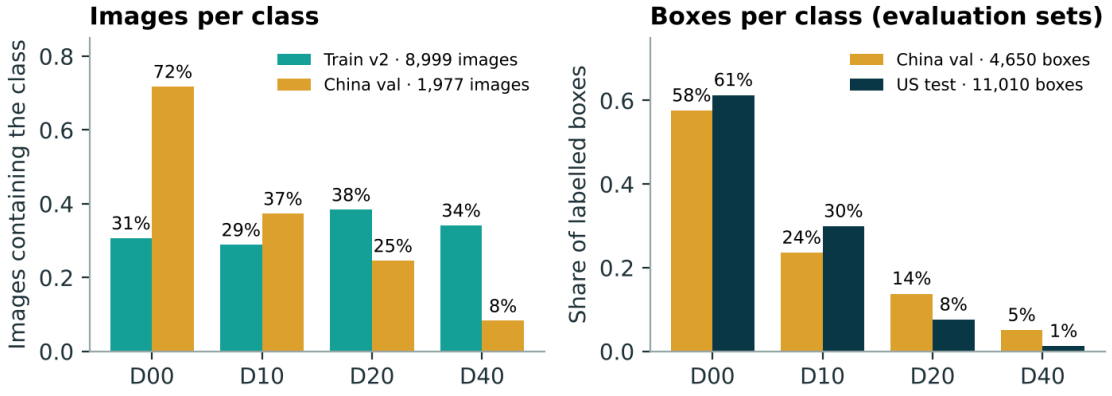


Figure 1. Class imbalance. Training was enriched so that each class appears in 29 to 38 % of images, but the evaluation sets are dominated by longitudinal cracks (D00 is 58 % of China boxes and 61 % of US boxes) while potholes are only 5 % and 1 %. This is why we report per-class results everywhere and never rely on overall mAP alone.

Quality issues and limitations

- **Imbalance:** cracks dominate; the US test set has only 135 pothole boxes, so pothole metrics there have wide uncertainty.
- **Label granularity differs by country:** some annotators draw one long box per crack, others many small boxes. This changes IoU matches and lowers measured recall without the model being "wrong" (Section 7).
- **Background images:** 1,427 training images (16 %) have no kept boxes; they teach the model what a healthy road looks like.
- **No GPS in RDD2022.** The duplicate experiment therefore uses **real EXIF GPS from our own photos**; nothing is simulated.
- **Reused development set:** China was used repeatedly for model and threshold choice, so its numbers are optimistic. The published checkpoint we started from may also have seen China images (overlap unknown).
- **Local set exposure:** the local photos were inspected during the project (error review), so they are diagnostic evidence rather than a perfectly untouched final holdout. No model or threshold was tuned on them.
- **Residual leakage risk:** one exact duplicate image was removed from the US cohort; near-duplicates across countries were not exhaustively searched.

3. System architecture

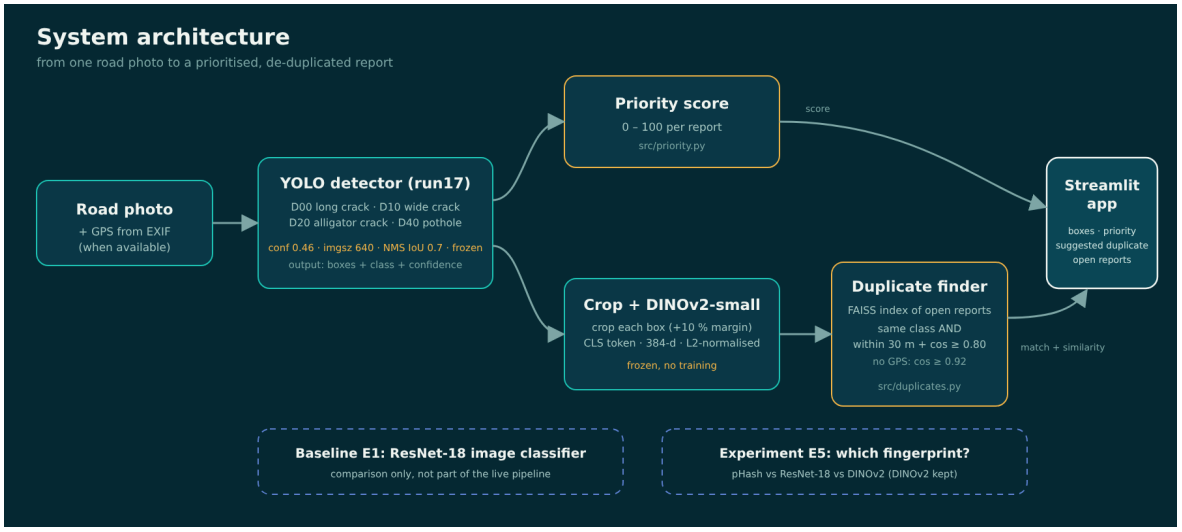


Figure 2. RoadScan pipeline. The detector runs on the original photo; faces and plates are blurred before anything is shown or stored. The ResNet-18 baseline and the pHash/ResNet embeddings are used only for comparison (E1, E5), not in the live path.

Component	What it does	Code
1. Damage detector	YOLOv8s (run 17) returns boxes, class D00/D10/D20/D40 and confidence. Frozen: confidence 0.46, image size 640, NMS IoU 0.7, max 300 boxes.	`src/roadscan_detector.py` , `configs/model_config.json`
2. Privacy blur	OpenCV face and licence-plate detectors; matching regions are Gaussian-blurred in every displayed or saved image.	`app/privacy.py`
3. Priority score	Documented rule-based formula, 0 to 100, from type, size, count and confidence (Section 4.3).	`src/priority.py`
4. Duplicate finder	Each box is cropped (+10 % margin) and embedded with frozen DINOv2-small (384-d CLS token, L2-normalised). Vectors are stored in a FAISS `IndexFlatIP` (inner product = cosine) with metadata (report id, class, GPS). A new report is a likely duplicate if: same class, within 30 m, cosine ≥ 0.80 (no GPS: ≥ 0.92).	`src/duplicates.py`
5. Maintenance app	Streamlit app: citizen reporting, municipality queue sorted by priority, report details and status workflow, damage map, technical pages showing per-step CPU timing.	`app/`

4. Baseline and main approach

4.1 Baseline: image-level multi-label classifier

ResNet-18 pretrained on ImageNet, final layer replaced by 4 sigmoid outputs ("which damage types are present?"), binary cross-entropy loss, 384 × 384 input, Adam (lr 1e-3), 8 epochs, seed 42. The backbone is frozen and only the head (2,052 parameters) is trained, the standard transfer-learning baseline. Per-class decision thresholds were chosen on the China development set. Two versions: v1 (600 images) and v2 (8,999 images). *Why this baseline:* it is the simplest model that answers the same question without localisation, so the gap to the detector measures what localisation adds (E1).

4.2 Main model: fine-tuned YOLO detector

We tried two routes (full log in Appendix B). Training YOLO11s from COCO weights on our 8,999 images (run 14) reached only 0.227 mAP@0.5 on China. A YOLOv8s checkpoint already trained on RDD2022 (SreekarAditya/yolo-rdd2022-benchmark) reached 0.962 without further training (run 15). Fine-tuning it with the default recipe (run 16) damaged it (0.794). A **gentle fine-tune** with a low learning rate (lr0 = warm-up bias lr = 0.0001, AdamW, 640 px, batch 16, seed 42, patience 5; 7 epochs, best epoch 2) kept its knowledge: **run 17, mAP@0.5 = 0.959, mAP@0.5:0.95 = 0.658**.

Run	Initialisation and training	mAP@0.5	mAP@0.5:0.95
14	YOLO11s from COCO, trained by us on v2	0.227	0.087
15	Published RDD2022 YOLOv8s, no training (reference)	0.962	0.671
16	Run 15 fine-tuned with run 14's recipe	0.794	0.447
17	**Run 15 gently fine-tuned (lr 0.0001) · adopted**	**0.959**	**0.658**

Why we adopted run 17. It contains our own fine-tuning on our sample (a brief requirement) while staying within 1.3 points of the published model. We state clearly that it does **not** beat the published starting point on China; the gentle recipe's contribution is that it adapts without forgetting, unlike run 16. A lower learning rate is used in fine-tuning precisely so that large gradient steps do not overwrite pretrained features.

4.3 Confidence threshold (justified, not default)

The rule was fixed *before* looking at the results: on China development data, choose the confidence that maximises the mean box F1 of the four classes, **subject to pothole recall ≥ 90 %**, because a missed pothole is

more costly than a false alarm. Grid 0.01 to 0.95; boxes matched one-to-one at IoU ≥ 0.5. The selected value is **0.46**.

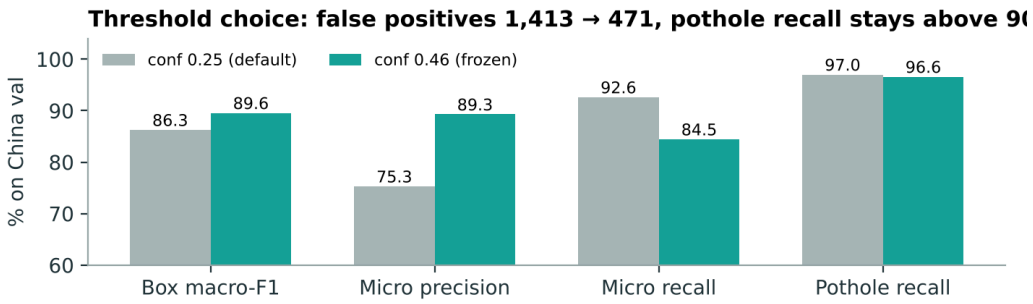


Figure 3. Moving from the Ultralytics default 0.25 to 0.46 cuts false positives by two thirds (1,413 → 471) while pothole recall stays at 96.6 %. The cost is more missed cracks (false negatives 343 → 723), an accepted trade-off because cracks are low priority.

4.4 Priority score

```
priority = 100 × (0.45·type + 0.20·size + 0.20·count + 0.15·confidence)
type       = highest class weight present: D40 pothole 1.0 · D20 alligator 0.8 · D10 transverse 0.5 · D00 longitudinal 0.4
size       = total box area ÷ image area ÷ 0.25   (capped at 1; damage covering a quarter of the photo is "large")
count      = number of boxes ÷ 5                  (capped at 1; five or more damages is "many")
confidence = mean detector confidence
```

Term	Justification
Type 0.45	Largest weight: potholes are a direct risk to vehicles and people; alligator cracking is the stage just before potholes form; isolated hairline cracks are the least urgent. The type weights follow this ordering, as the brief recommends.
Size 0.20	Larger damaged area means more repair material and faster spread.
Count 0.20	Several damages in one place indicate a failing road section. Capped at 5 so that one photo with many small cracks cannot dominate.
Confidence 0.15	Smallest weight so an uncertain detection cannot push a report to the top on its own.

Bands: **70 to 100 Urgent** (repair this week) · **40 to 69 Schedule** · **0 to 39 Monitor**. The bands are a proposal to be agreed with maintenance staff.

Sensitivity check (±20 %). We changed each weight by −20 % and +20 % (8 variants) and recomputed the scores. On three worked examples (thin crack 32.6, one pothole 70.5, five potholes 99.2) the ranking never changed. We then repeated the check on 200 randomly generated reports (1 to 6 boxes each, realistic class mix, seed 42):

Weight changed	−20 %: Spearman ρ	max rank shift	top-10 kept	+20 %: Spearman ρ	max rank shift	top-10 kept
Type	0.998	18 / 200	10 / 10	0.998	12 / 200	10 / 10
Size	0.999	12	10 / 10	0.999	11	10 / 10
Count	0.999	10	10 / 10	0.999	9	10 / 10
Confidence	1.000	5	10 / 10	1.000	4	10 / 10

Interpretation. The ordering is robust (ρ ≥ 0.998, the top 10 never changes). What *does* move is the band of borderline reports: 2 to 14 of 200 reports change band, and the single-pothole example (70.5) drops to Schedule when the type weight falls by 20 %. The weights therefore matter little for the ranking but matter for

reports near a band edge, which is why band cut-offs should be set with the municipality. The same check runs live on real photos in the app's *Priority Score* page (script: `scripts/priority_sensitivity.py`).

5. Experiments E1 to E5

Each experiment changes one thing, is reported as a table or chart, and ends in a decision for the final system.

E1. Baseline classifier vs detector: what does localisation add?

To compare fairly, the detector's boxes are turned into image-level answers ("is D40 present?" = any D40 box above threshold) and both models are scored with the same per-class image F1 on the **same 1,977 China images**.

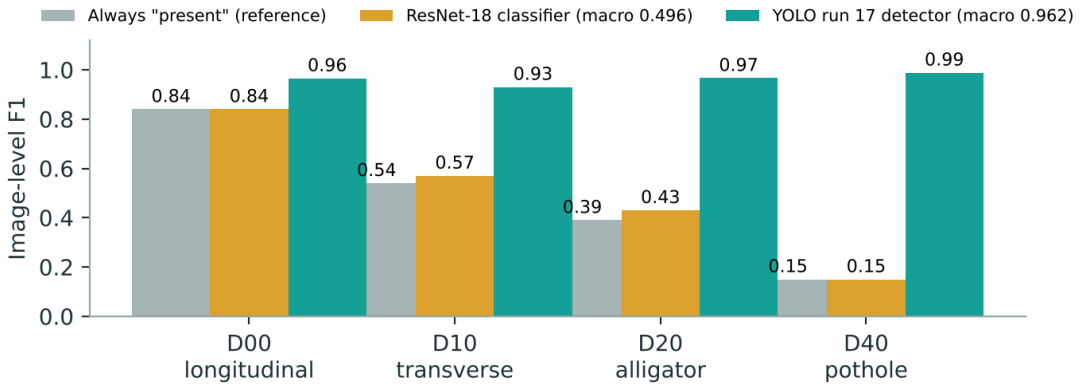


Figure 4. Image-level F1 per class. Grey = a trivial model that always says "present".

Metric (China development, 1,977 images)	ResNet-18 v2	YOLO run 17
Image macro-F1	0.496 (thresholds tuned on these images)	0.962 (fixed 0.5)
AUROC D00 / D10 / D20 / D40	0.52 / 0.65 / 0.65 / 0.46	0.998 / 0.996 / 0.998 / 1.000
US held-out, image macro-F1	0.362	n/a (box metrics in E4)

Interpretation. The classifier is barely better than always answering "yes": its AUROC is close to 0.5 and its pothole F1 (0.15) equals the trivial reference. Damage occupies a small part of a road photo, so a global image feature is dominated by asphalt, sky and cars; the detector looks at each region separately. The detector is better by **+0.47 macro-F1** even though the comparison favours the classifier (its thresholds were tuned on these images). The gap is likely overstated because run 17's starting checkpoint may have seen China.
Decision: localisation is worth its complexity, and it is also required downstream: the priority score needs box size and count, and the duplicate finder needs crops.

E2. Input resolution (640 vs 960): accuracy vs CPU speed

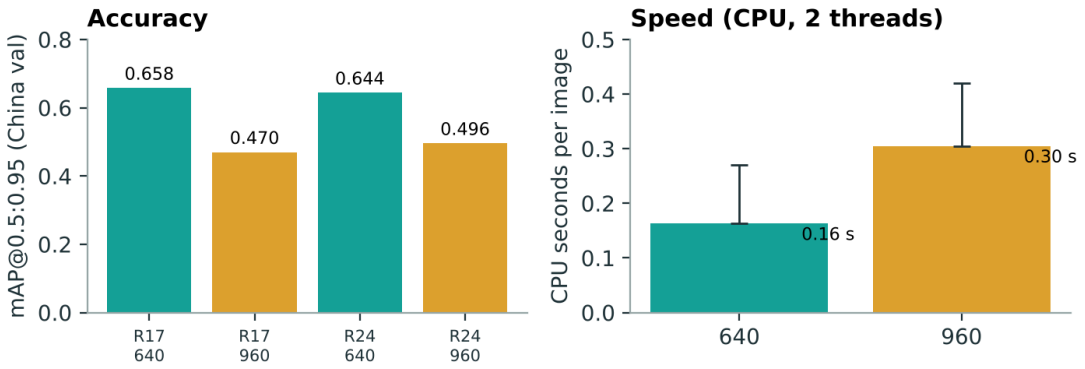


Figure 5. Left: mAP@0.5:0.95 on China (runs 17 and 24). Right: CPU time per image for run 17 (median of 30 timed predictions, 15 images, 2 threads, 3 warm-up runs; bar to the 95th percentile).

Setting	mAP@0.5	mAP@0.5:0.95	CPU s / image (median)
Run 17 at 640 (adopted)	**0.959**	**0.658**	**0.163**
Run 17 evaluated at 960	0.869	0.470	0.305
Run 24: run 17 re-fine-tuned at 960, evaluated at 960	0.882	0.496	≈ 0.31
Run 25: YOLO11m from COCO, at 640 / 960	0.262 / 0.225	0.101 / 0.079	n/m

Interpretation. Going to 960 nearly doubles CPU time (×1.9) and *lowers* accuracy, even after re-training at 960 (run 24). A larger input mainly helps thin cracks when the model was trained at that scale on enough data; our checkpoint's features are tuned to 640 and most RDD images are themselves only 512 to 720 px, so upscaling adds cost without new detail. The bigger YOLO11m trained from COCO (about 6.5 h) stayed far below the RDD-initialised small model: initialisation mattered more than size. Early nano vs small pilots (runs 3, 4) were on an older split and are listed in Appendix B only. **Decision:** keep YOLOv8s at 640.

E3. Augmentation: which settings help, and which are unrealistic?

Run 20: four arms from the same initialisation, data, seed and recipe, 7 epochs each, removing one augmentation group at a time.

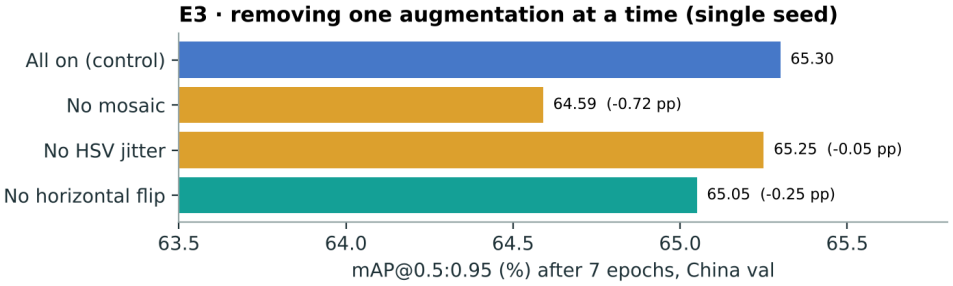


Figure 6. mAP@0.5:0.95 after 7 epochs on China development data (single seed).

Interpretation. Mosaic helps most (−0.72 pp when removed): stitching four images varies scale and context, which matters for small potholes. Horizontal flip helps a little (−0.25 pp): a left-right mirrored road is still a realistic road (crack direction relative to the lane is preserved). HSV jitter is within noise (−0.05 pp) though mild jitter is physically plausible (sun, shade, asphalt colour). **Vertical flip is kept at zero** because it puts the sky at the bottom and the road at the top, which never happens in a forward-facing road photo, and it would also swap the perspective of near and far damage. Extreme hue shifts are likewise unrealistic. Caveats: one seed and a short horizon, so differences under ~0.3 pp are not reliable. **Decision:** keep the default recipe (mosaic, mild HSV, horizontal flip on; vertical flip off).

E4. Cross-country generalisation and local photos

Trained on Japan, India and Czech; evaluated with frozen settings on China (development), the USA (held-out country) and our own city.

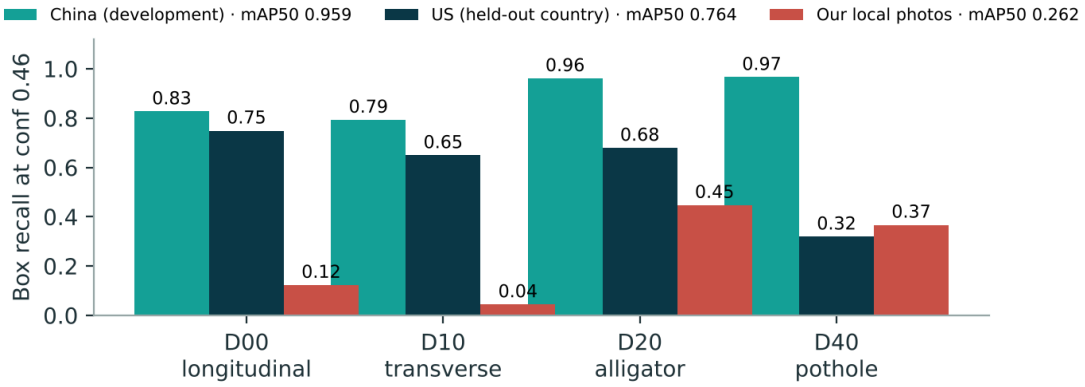


Figure 7. Box recall per class at confidence 0.46 on three domains.

Domain	Images	Boxes	mAP@0.5	mAP@0.5:0.95	Precision	Recall	Pothole recall
China (development)	1,977	4,650	0.959	0.658	0.893	0.845	0.966 (227/235)
USA (held-out country)	4,804	11,010	0.764	0.497	0.821	0.709	0.319 (43/135)
Our city (team-built)	**190**	**333**	**0.262**	**0.131**	**0.500**	**0.174**	**0.367 (18/49)**

Interpretation. The domain-shift gap in mAP@0.5 is **−0.20** for a new country and **−0.70** for our city. The classes suffer differently: on our roads transverse cracks (D10, recall 4 %) and longitudinal cracks (D00, 12 %) almost disappear, while alligator cracking (45 %) and potholes (37 %) survive better because they are larger, textured regions. In the USA potholes suffer most (32 %), partly because US potholes are labelled as small fragmented boxes. Likely causes for our city: different asphalt colour and texture, strong sun and hard shadows, phone photos taken on foot at varied angles (RDD photos are mostly taken from a moving vehicle), and annotation style. **Decision:** we do not claim local readiness; the fix is local data (Section 10).

E5. Duplicate embeddings: pHash vs ImageNet ResNet vs DINOv2

Each photo's detected boxes are cropped and embedded by three methods; pairs are scored by cosine similarity (Hamming similarity for pHash). Thresholds were tuned on the **10 tuning pairs** only and measured once on **9 different test pairs** (24 different-damage combinations). GPS is switched off here so only the embedding is compared.

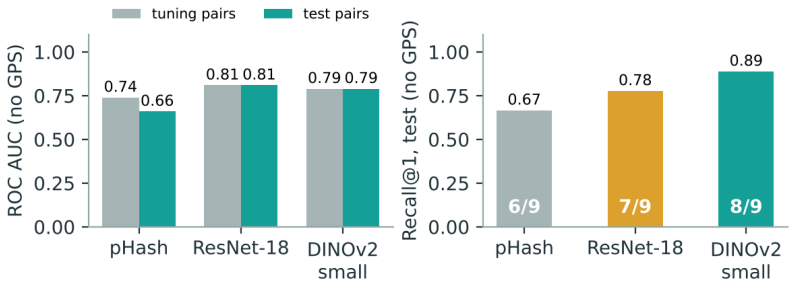


Figure 8. AUC and test Recall@1 without GPS.

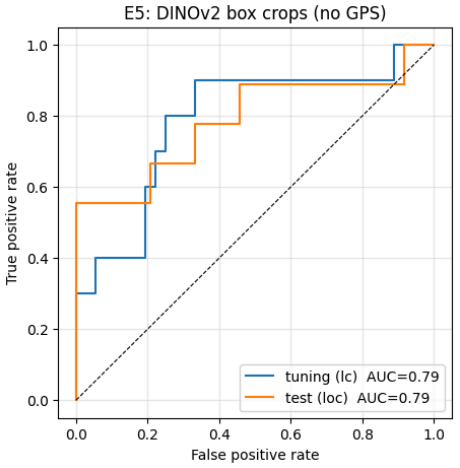


Figure 9. ROC of DINOv2 crop similarity, tuning vs test pairs.

Method (no GPS)	AUC tuning / test	Recall@1 tuning / test	Recall@5 tuning / test
DINOv2-small (adopted)	**0.79 / 0.79**	**7/10 · 8/9**	**10/10 · 9/9**
ResNet-18 (frozen ImageNet)	0.81 / 0.81	8/10 · 7/9	10/10 · 9/9
pHash	0.74 / 0.66	4/10 · 6/9	10/10 · 9/9

Interpretation. pHash is clearly weakest and drops most on test: it compares low-frequency pixel patterns, which change with viewpoint, lighting and crop. DINOv2 and ResNet-18 are tied within noise on 19 pairs. We keep DINOv2 because self-supervised training (matching different views of the same image) makes its features describe local texture and shape rather than an ImageNet object label, which is the property near-duplicate retrieval needs; it also had the best test Recall@1 and stable AUC. A larger pair set is needed to separate the two.

Adding location. Embeddings alone are not enough: at 0.80 without GPS, 12 of 24 different damages would be merged. With the 30 m GPS filter:

Setting	Tuning: caught / false	Test: caught / false	Test precision / recall
Same class + within 30 m + cosine ≥ 0.80 (main path)	**9/10 · 4/36**	**8/9 · 0/24**	**1.00 / 0.89**
No GPS, cosine ≥ 0.80	9/10 · 16/36	8/9 · 12/24	0.40 / 0.89
No GPS, cosine ≥ 0.92 (fallback)	4/10 · 3/36	4/9 · 0/24	1.00 / 0.44

Cost of each error. A missed duplicate wastes one crew visit; a false duplicate *hides a real report* from the queue. We therefore chose strict settings that accept missed duplicates to keep false merges low: the chosen setting (same class + within 30 m + cosine ≥ 0.80) gave 0 of 24 false merges on the 9 test pairs, but 4 of 36 on the 10 tuning pairs, so zero false merges holds on the test pairs only. Because false merges are still possible, the app only *suggests* a duplicate and staff confirm it. Radius 30 m and 40 m gave identical results on tuning pairs, so the tighter 30 m was kept.

6. Evaluation on the held-out test set and the team-built evaluation set

6.1 Detection metrics on development data (run 17, China)

Class	Labels	AP@0.5	AP@0.5:0.95	Precision @0.46	Recall @0.46	F1 @0.46
D00 longitudinal crack	2,678	0.940	0.662	0.891	0.827	0.858
D10 transverse crack	1,096	0.929	0.604	0.861	0.793	0.826
D20 alligator crack	641	0.981	0.734	0.929	0.961	0.945
D40 pothole	235	0.986	0.632	0.942	0.966	0.954
All (mAP / macro)	**4,650**	**0.959**	**0.658**	**0.906**	**0.887**	**0.896**

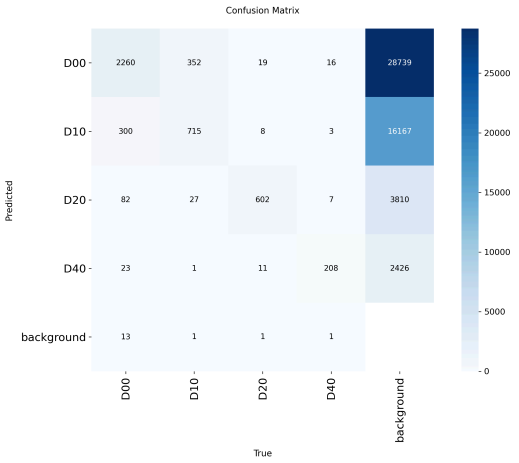


Figure 10. Ultralytics confusion matrix (columns = true class). It is computed at the validation confidence floor, so the "background" column (false alarms) is far larger than at our 0.46 operating point. The useful reading is the off-diagonal: D00 and D10 are confused with each other (352 and 300 boxes), i.e. crack *orientation* is the hardest decision.

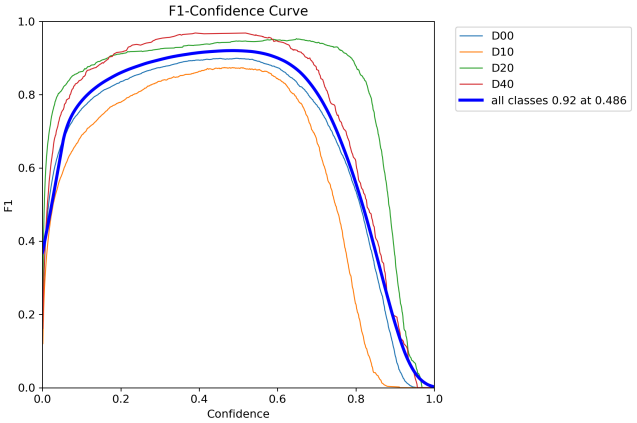


Figure 11. F1 vs confidence per class. The curves are flat between about 0.3 and 0.6, so 0.46 sits on the plateau; potholes and alligator cracks keep high F1 at every threshold while cracks fall first.

Per-class reading and class imbalance. The rarest class (potholes, 5 % of boxes) has the highest AP@0.5 because potholes are compact, high-contrast objects. The most frequent class (D00, 58 %) has the most errors in absolute terms (463 missed boxes) because cracks are thin and their box extent is ambiguous. AP@0.5:0.95 is lower for D40 than D20 because exact pothole edges are hard to match at strict IoU. Overall mAP would hide all of this.

6.2 Held-out country (USA, measured once)

Class	Labels	AP@0.5	AP@0.5:0.95	Precision	Recall	F1
D00	6,747	0.868	0.583	0.819	0.749	0.783
D10	3,294	0.799	0.459	0.804	0.649	0.719
D20	834	0.854	0.606	0.906	0.679	0.776
D40	135	0.536	0.341	0.768	0.319	0.450

6.3 Team-built local test set (our city, 190 images)

Class	Images	TP	FP	FN	Precision	Recall	Image-level F1
D00	89	19	22	136	0.463	0.123	0.417
D10	53	4	15	87	0.211	0.044	0.200
D20	33	17	9	21	0.654	0.447	0.724
D40	43	18	12	31	0.600	0.367	0.667
All	**134 + 56 healthy**	**58**	**58**	**275**	**0.500**	**0.174**	**0.502 (macro)**

Image-level F1 is much higher than box recall (0.67 vs 0.37 for potholes): the model usually notices *that* a damaged road has a pothole even when it misses individual boxes. For the supervisor's queue, the image-level view is what decides whether a report is flagged. Lowering the threshold is not a free fix: an alternative model (run 23) at confidence 0.001 found 44 % of boxes but raised 16,740 false boxes and alarmed on all 56 healthy roads (Appendix B).

6.4 Duplicate finder (team-built pairs)

Recall@1 = 9/9 and Recall@5 = 9/9 on test pairs with GPS (8/10 and 9/10 on tuning); this is a ranking measure: the true match is the first candidate after GPS filtering. The decision at the fixed threshold (cosine ≥ 0.80 within 30 m) flags 8 of 9 test duplicates: precision 1.00, recall 0.89. These are two different measures, so they do not contradict each other: one true match ranked first but its similarity was below 0.80. ROC curve in Figure 9. 11 of the 30 collected pairs were unusable because the detector found no box in at least one photo, so the duplicate finder inherits the detector's local recall problem.

6.5 System latency on CPU

Measured on our team's demo photos (9 phone photos resized to 1,600 px), 2 CPU threads, warm model:

Step	Median time	Note
Open, EXIF-rotate and resize photo	0.35 s	JPEG draft decoding of 3 to 10 MB phone photos
YOLO detection (640)	0.21 s	0.16 s on the 15-image benchmark in E2; 0.19 s in the run 17 Kaggle CPU test
Face and plate blur	1.08 s	The slowest step; Haar cascades at 1,280 px
Priority score	< 0.001 s	Pure arithmetic
Sum of the steps above	**1.70 s**	**Median over the 9 photos (range 1.46 to 2.44 s; the first photo includes model warm-up)**
+ DINOv2 crops and FAISS search, saving	app panel	Timed per report in the app's timing panel; grows with the number of boxes. End to end we observe about 2 to 3 s per photo in the deployed app

The bottleneck is privacy blurring, not the AI model; a lighter face/plate detector would roughly halve end-to-end time.

7. Error analysis

Failures were grouped automatically by geometry (IoU against labels at confidence 0.46) and then reviewed by hand. Counts below are for the adopted run 17.

Category	Count	Example	Hypothesis	Possible fix
1. Thin cracks not detected (local)	169 of 275 local misses have no box of the same class nearby; 223 of 275 are D00/D10	Faded, sealed or hairline cracks (Fig. 12a)	Domain shift in asphalt texture and lighting; thin cracks shrink to 1 to 2 px at 640	Fine-tune on part of our labelled photos; tiling for large photos
2. Box extent disagreement	45 local; 494 US FN, 564 US FP	One long predicted box over several small labels (Fig. 12c)	Annotation granularity differs between countries and annotators, so IoU < 0.5 even when the damage is found	Consistent labelling guide; image-level or merged-box metrics for the queue
3. Low-confidence correct boxes	41 local	Shadowed pothole at 0.47 to 0.56 (Fig. 12b)	Hard shadows and glare reduce confidence just around the threshold	Local fine-tuning; HSV and brightness augmentation tuned for strong sun
4. Crack orientation confusion (D00 ↔ D10)	20 local; 352 + 300 on China (matrix)	Diagonal crack labelled D00, predicted D10	Orientation is relative to the lane, which the model must infer; diagonal cracks are ambiguous	Merge D00/D10 as "linear crack" for priority (they share low weights anyway)
5. Duplicate boxes on one damage	127 China FP, 110 US FP	Two D40 boxes on one pothole (Fig. 12b)	NMS at IoU 0.7 keeps partially overlapping boxes of the same object	Lower NMS IoU for D40 or merge overlapping same-class boxes before scoring
6. Missed duplicates	1 test, 1 tuning	loc26: same pothole, similarity 0.39 at 1.5 m; lc2: 0.54 at 31.8 m	Very different viewpoint; or GPS error pushes the pair just outside 30 m	Multi-crop embeddings; adaptive radius from GPS accuracy
7. False duplicates without GPS	12 of 24 test at 0.80	Different potholes 243 m to 5 km apart look alike	Generic asphalt texture; many potholes look similar	Always require location; 0.92 when GPS is missing
8. Blur false positives	4 of 9 demo photos	Building wall patch blurred (Fig. 12c)	Haar cascades fire on window and wall textures	Harmless for privacy but costly in time; a CNN face/plate detector



Figure 12. Real failures of the adopted model on our own photos (faces and plates blurred). (a) Faded longitudinal and transverse cracks: no box at all. (b) A shadowed pothole receives two overlapping boxes at 0.56 and 0.47, close to the threshold. (c) A thin crack is covered by one wide D00 box much larger than the label; a wall region was blurred by mistake.

What the categories tell us. On our roads the dominant problem is *missed thin cracks* (category 1), not wrong boxes: precision is 0.50 but recall 0.17. Since cracks carry the lowest priority weights, the practical harm is smaller than the raw recall suggests, but missed potholes (31 of 49) are serious and are the first target for local fine-tuning.

8. Application: how it works, latency and artifact loading

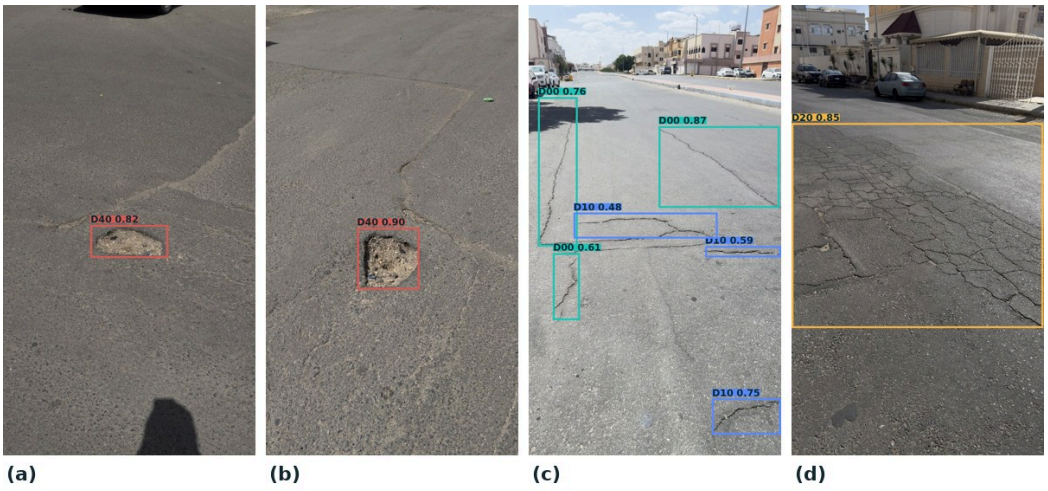


Figure 13. Output of the deployed pipeline on team photos. (a, b) The same pothole photographed twice: D40 at 0.82 and 0.90, priority 63.1 and 65.2; the second upload is the duplicate-demo case. (c) Six cracks, priority 70.0 (Urgent by count). (d) Alligator cracking at 0.85, priority 72.7.

8.1 User flow

1. **Citizen → Report damage.** Upload a photo (JPEG, PNG or iPhone HEIC). The app corrects EXIF rotation, resizes to at most 1,600 px, runs YOLO, blurs faces and plates, and shows boxes, class, confidence, priority score and band. GPS is read from EXIF (locations outside Saudi Arabia are rejected); if the photo has no GPS, the device location or a click on the map is used.
2. **Duplicate check.** Crops are embedded with DINOv2 and searched in FAISS among *open* reports. A match shows "likely duplicate of RS-xxxx" with similarity and distance; otherwise "new report".
3. **Municipality → Queue.** Staff (access code) see all open reports sorted by priority, open a report's details (evidence, map, detections, timeline), assign a team, set a repair date and change status (new → review → scheduled → in progress → repaired, or rejected).
4. **Technical pages** for the panel: per-step CPU timing, batch map of many photos with GeoJSON export, local-test error browser, model results and the live priority sensitivity check.

8.2 Artifact

Item	Where and how it is versioned
Detector weights	`run17_best_finetuned.pt` in `app/models/` and GitHub release `v2-yolo`; SHA-256 recorded in `configs/model_config.json`
Preprocessing and thresholds	`configs/model_config.json`: class map, imgsz 640, conf 0.46, NMS 0.7, max_det 300 (loaded by the detector class, so code and settings cannot drift)
Duplicate thresholds	`src/duplicates.py`: radius 30 m, 0.80, 0.92 (constants tuned in E5)
Vector index	FAISS index persisted as `vectors.npy` + `meta.json` and rebuilt at start-up
Baseline	Releases `v1-baseline`, `v2-baseline`; `artifacts/download_weights.py`
Tests	`tests/test_roadscan_detector.py`: class mapping, frozen defaults equal the config, empty images, input errors

Loading in a fresh process with one command:

```
python -c "from src.roadscan_detector import RoadDamageDetector as R;
print(R('app/models/run17_best_finetuned.pt', device='cpu').detect_damage('road.jpg'))"
```

9. Responsible AI

Privacy

- Road photos can contain faces and licence plates. The app blurs both before any image is displayed, saved as a thumbnail or used for a duplicate crop; the detector itself runs on the unblurred image in memory only.
- Our OpenCV cascades are fast but imperfect: they miss side faces and angled Saudi plates and sometimes blur walls (Section 7). **In production** a stronger CNN face/plate detector should run on the server, originals should be deleted after processing, only blurred copies kept, access limited by role, and retention periods defined under the Saudi Personal Data Protection Law (PDPL).
- GPS reveals where a citizen stood. We store the damage location, not the reporter's identity, and the public views show reports, not people.

Fairness: under-reported neighbourhoods

- A queue driven only by citizen reports favours areas whose residents report more (more smartphones, more time, more trust). Damage in under-reporting neighbourhoods would rank as "no reports" rather than "no damage".
- The model is also less accurate on roads unlike its training data; neighbourhoods with older or patched asphalt may be systematically under-detected.
- **Mitigations:** combine citizen reports with planned inspection-vehicle routes that cover every district; monitor reports per district on the analytics map and inspect districts with unusually few reports; never use "zero reports" as evidence of good roads.

Misuse

- **Gaming the queue** by uploading many photos of the same damage: the duplicate finder groups them and the count term is capped at 5.
- **Fake or irrelevant reports:** photos with no detection receive priority 0 and stay at the bottom; locations outside the country are rejected.
- **Surveillance:** the system must not be used to identify people or vehicles; blurring and storage rules above enforce this.

Limits of the system for the real user

- The detector misses most thin cracks and about two thirds of pothole boxes on our local roads (Section 6.3). It must be presented to staff as decision support, not as an inspector.
- **The final decision stays with a person:** staff confirm duplicates, can change priority and status, and a crew inspects before repair.
- GPS in our duplicate experiment is **real EXIF GPS**, not simulated. Phone GPS error (≈ 5 to 30 m) can still split one damage into two reports; a missing GPS falls back to the stricter 0.92 threshold, which catches fewer duplicates.
- Duplicate results rest on 19 pairs and the E3 ablation on one seed; both need more data before strong claims.

10. Conclusion and next steps

Main result. RoadScan turns a road photo into blurred evidence, typed damage boxes, a justified priority score and a duplicate check in about 2 to 3 seconds on CPU. The detector beats the image-level baseline by a large margin (0.962 vs 0.496 macro-F1), the threshold was chosen for the cost of a missed pothole, the priority ranking is stable under ±20 % weight changes, and the duplicate finder made 0 of 24 false merges on

the 9 test pairs (4 of 36 on the 10 tuning pairs) with the chosen setting (same class + within 30 m + cosine ≥ 0.80).

Main limitation. Domain shift. mAP@0.5 falls from 0.96 (development) to 0.76 (new country) to 0.26 (our city), with thin cracks almost invisible to the model on local roads. Lowering the threshold does not fix this.

Next steps, in order:

1. Fine-tune on a portion of our labelled local photos while keeping a *new, untouched* local test set, and measure how much of the gap closes (the brief's distinction-level extension).
2. Tiling or 960 px inference trained at that scale for thin cracks, chosen on development data and measured once locally.
3. Collect 50+ duplicate pairs to separate DINOv2 from ResNet-18 and to tune radius by GPS accuracy.
4. Replace Haar cascades with a CNN face/plate detector; agree priority bands with maintenance staff.
5. Process dashcam video (sample frames, track damage across frames) for inspection vehicles.

What we would do differently. Collect and label local photos at the start, so that every model decision could have been checked against our own roads earlier, and fix a single labelling guide for crack box extent before annotating.

11. Contribution statement

Member	Main ownership	Also contributed
Sarah Al-Bahesh (saravii7)	Data pipeline: RDD2022 download, cleaning and conversion to YOLO, country split, v1 and v2 sampling (seed 42), Kaggle datasets and manifests; local and clean-road photo collection with the team; Streamlit application end to end (citizen and municipality flows, privacy blur, duplicate integration, map, timing pages); repository structure, README, integration and deployment	E2 CPU latency measurement, demo scenario, this report
Rimas Al-Abunasser	YOLO detector: runs 3 to 28, gentle fine-tuning (run 17), confidence threshold selection, E2, E3 augmentation ablation, E4 country and local evaluation, error categories, local test set labelling and review	Detector module, tests and model configuration
Fajer Alshahrani	ResNet-18 baseline v1 and v2 (E1), priority formula and sensitivity check, duplicate finder with DINOv2 and FAISS, duplicate pair collection, E5 three-method comparison, GPS radius and no-GPS threshold	Duplicate module used by the app

All three members reviewed each other's components before submission so that any member can explain any part of the system.

References

1. Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Sekimoto, Y. (2022). *RDD2022: A multi-national image dataset for automatic road damage detection.* arXiv:2209.08538. Data: figshare, doi:10.6084/m9.figshare.21431547, licence CC BY 4.0.
2. SreekarAditya. *yolo-rdd2022-benchmark* (YOLOv8s RDD2022 checkpoint, `yolov8s_seed0_best.pt`). huggingface.co/SreekarAditya/yolo-rdd2022-benchmark. Initialisation of our run 17.
3. Jocher, G., Qiu, J., Chaurasia, A. *Ultralytics YOLO* (YOLOv8 and YOLO11). github.com/ultralytics/ultralytics, AGPL-3.0.
4. Lin, T.-Y. et al. (2014). *Microsoft COCO: Common objects in context.* ECCV.
5. He, K., Zhang, X., Ren, S., Sun, J. (2016). *Deep residual learning for image recognition.* CVPR. (ResNet-18 baseline and E5 embedding)
6. Deng, J. et al. (2009). *ImageNet: A large-scale hierarchical image database.* CVPR.
7. Oquab, M. et al. (2023). *DINOv2: Learning robust visual features without supervision.* arXiv:2304.07193. Model `facebook/dinov2-small`.
8. Johnson, J., Douze, M., Jégou, H. (2019). *Billion-scale similarity search with GPUs.* IEEE Transactions on Big Data. (FAISS)
9. Zauner, C. (2010). *Implementation and benchmarking of perceptual image hash functions.* Master's thesis, Upper Austria University of Applied Sciences. Implementation: `imagehash` Python library. (pHash)
10. Viola, P., Jones, M. (2001). *Rapid object detection using a boosted cascade of simple features.* CVPR. Implementation: OpenCV Haar cascades. (Face and plate blurring)
11. Saudi Data and AI Authority (SDAIA). *Personal Data Protection Law* (PDPL), Royal Decree M/19, 2021, and its implementing regulations.
12. Olo Academy (2026). *AI Engineering Bootcamp (AIEP) Capstone Project Brief*, Track A: RoadScan.

Appendix A. Reproducibility

```
git clone https://github.com/saravii7/3-EyeRoadScanProject.git
cd 3-EyeRoadScanProject
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install -r app/requirements.txt
python -m unittest discover tests                          # smoke tests of the detector contract
streamlit run app/Home.py                                  # the demo
python scripts/e2_cpu_latency.py                           # E2 CPU timing
python scripts/priority_sensitivity.py                     # ±20 % sensitivity check
```

Claim in this report	Source file
Run 14 to 17 scores	`results/run14` … `run17`, `docs/reports/RUN16_COMPARISON.md`, `RUN17_COMPARISON.md`
Threshold 0.46	`results/run19/threshold_selection_results.json`, `docs/reports/THRESHOLD_SELECTION.md`
E1 detector image-level F1 and AUROC	`results/run17/17-roadscan-gentle-finetuning-results.json`
E1 baseline	`notebooks/baseline/02-baseline.ipynb`, `04-baseline-v2.ipynb`
E2 accuracy / CPU time	`results/run28/run24_table_02.csv`, `run25_table_01.csv`, `results/e2/cpu_latency.json`
E3	`docs/reports/E3_AUGMENTATION.md`
E4 countries / local	`results/run21/country_comparison.csv`, `country_per_class.csv`, `results/run28/run22_table_*.csv`
E5 and duplicate thresholds	`notebooks/05-duplicates.ipynb`
Full YOLO history	`notebooks/yolo/28-roadscan-yolo-journey-summary.ipynb`, `docs/reports/YOLO_CLOSEOUT.md`

Seeds are fixed at 42 for sampling, training and the sensitivity simulation. No secrets are stored in the repository; the application needs none.

Appendix B. Experiment log (baseline, and YOLO runs 3 to 28)

B.1 Baseline classifier (ResNet-18)

Attempt	What happened	What we changed
v1, 600 images, frozen backbone, 5 epochs, threshold 0.5	F1 = 0 for D00, D10 and D40 (the model never said "present"); validation AUC 0.41 to 0.61, i.e. at or below chance on China	Checked that the model learned at all: train AUC 0.69 to 0.80, so it learned training data but did not transfer
v1, whole network unfrozen	Train loss fell to 0.06 while validation loss rose to 1.27: strong over-fitting on 600 images	Went back to a frozen backbone
v1, with vs without augmentation	Validation AUC changed by at most 0.01 per class	Kept the simple version
v1, hold-out images from the training countries	AUC per class 0.60 / 0.69 / 0.77 / 0.86 on unseen images from the same countries vs 0.38 / 0.68 / 0.55 / 0.51 on China: the baseline also suffers domain shift	Evidence used in E4 discussion
v1, per-class thresholds tuned on China	Macro-F1 0.482, but only by predicting "present" for almost every image (equal to the trivial reference)	Needed more data → v2
v2, 8,999 images	Macro-F1 0.496 on China, 0.362 on the US; AUC still 0.46 to 0.65	Final baseline for E1: image-level features cannot find small damage

B.2 YOLO detector

Run	What was tried	Outcome and decision
3	YOLOv8n on 600-image pilot	China mAP@0.5 0.062: pipeline works, data too small
4	Nano vs small and no-mosaic pilot	Old split, stopped by GPU memory after 3 arms; not used as final E2
6 to 8	Larger resolutions 768/960 on saved checkpoints	960 not automatically better
9 to 13	Data diagnosis, layer freezing, balanced sampling, regularised recipe	Full fine-tuning slightly better; balancing and regularisation did not help
14	YOLO11s from COCO on v2 (64 epochs)	0.227 mAP@0.5: training from COCO too weak with our budget
15	Published RDD2022 YOLOv8s, no training	0.962: strong reference
16	Run 15 fine-tuned, standard recipe	0.794: forgetting; rejected
17	Run 15 gently fine-tuned (lr 1e-4)	0.959: **adopted**
18	Inference helper (D-code mapping) checks	Verified class mapping on RDD samples
19	Confidence selection on China	0.46 frozen
20	E3 augmentation ablation	Keep default recipe
21	Country comparison and error categories	US gap −0.20 mAP@0.5; pothole recall 31.9 %
22	Runs 14 and 17 on 190 local images	Run 17: P 0.50, R 0.17
23	YOLO11s (C2PSA attention) vs YOLOv8s, 1.5 h each	Neither beat run 17
24	Run 17 re-fine-tuned at 960	0.496 mAP@0.5:0.95 at 960 vs 0.658 at 640: keep 640
25	YOLO11m at 960 (~6.5 h)	0.101 / 0.079: size does not replace good initialisation
26	Tiling on US	Failed before scoring (missing input); no result claimed
27	Run 23 on local set at 0.001 and 0.46	0.001: 16,740 FP, alarms on 56/56 healthy roads; 0.46: 1 healthy alarm, recall 9.6 %
28	Journey summary, 18 figures	Documentation only

Appendix C. Full country and class tables (run 17, confidence 0.46)

Cohort	Role	Images	mAP@0.5	mAP@0.5:0.95	Micro P	Micro R	FP / 100 img
China	Development (reused)	1,977	0.959	0.658	0.893	0.845	23.8
USA	Held-out country	4,804	0.764	0.497	0.821	0.709	35.5
Japan	Unused images, 2 classes only	500	0.823	0.505	0.842	0.643	11.6
India	Unused images, 42 boxes	500	0.620	0.403	0.575	0.548	3.4
Czech	Unused images, no labelled boxes	500	n/a	n/a	n/a	n/a	5.2

Japan, India and Czech complements are depleted of damage because the training sample was enriched for it; they are diagnostics, not representative country scores. US error categories: 2,618 missed with no detection above threshold, 494 localisation, 97 wrong class; 922 background/annotation-review FP, 564 localisation FP, 110 duplicate FP, 109 wrong class. 1,784 of 4,122 missed boxes across cohorts cover under 1 % of the image.

Error category on local photos (run 17)	D00	D10	D20	D40
Matched (true positive)	19	4	17	18
No same-class box nearby	96	56	5	12
Localisation or label extent (IoU < 0.5)	18	9	5	13
Correct box below 0.46	13	11	11	6
Overlapped by another class	9	11	0	0

Appendix D. Track A full-mark checklist

Brief item	Status	Where
Baseline classifier and fine-tuned detector compared	**Done**	4.1, E1
Per-class AP, precision, recall, and a justified confidence threshold	**Done**	4.3, 6.1
Local test set collected, labelled, and reported separately	**Done**	2, 6.3
Cross-country generalisation experiment with interpretation	**Done**	E4, 6.2
Documented priority formula with a sensitivity check	**Done**	4.4
Duplicate finder evaluated on team-built pairs with three embedding methods	**Done**	E5, 6.4
Working app on a panel-chosen photo, with CPU latency reported	**Done**	6.5, 8
Faces and plates blurred; GPS source stated (real EXIF, not simulated)	**Done**	9
Required experiments E1 to E5 with interpretation	**Done**	5
Vector database used for a real purpose (FAISS duplicate search)	**Done**	3, E5
Extension: GeoJSON export and map of the queue	**Done**	8.1
Extension: fine-tune on part of local data with a separate local test set	**Next step**	10

Appendix E. Concepts for Q&A

Question	Short answer in our project's terms
Classification vs detection vs segmentation	Classification says *what* is in the image; detection says what and *where* (boxes); segmentation labels every pixel. We need detection: the priority needs size and count, and the duplicate finder needs crops. E1 shows classification alone fails.
IoU and mAP@0.5	IoU = overlap area ÷ union area of a predicted and a true box. A prediction is correct if IoU ≥ 0.5 and the class matches. AP is the area under the precision-recall curve over all confidences for one class; mAP averages the 4 classes. mAP@0.5:0.95 averages over IoU 0.5 to 0.95.
Why NMS	YOLO predicts many overlapping boxes for one object; non-max suppression keeps the most confident and removes others with IoU > 0.7. Remaining duplicates (Section 7, category 5) are boxes that overlap less than that.
Transfer learning and low learning rate	We start from weights already trained on road damage and update them gently (lr 0.0001). A high learning rate overwrote the pretrained features (run 16 fell from 0.962 to 0.794). The baseline froze the whole ResNet backbone and trained only the head.
Valid augmentations	Horizontal flip, mild brightness/colour jitter and mosaic are realistic for road photos. Vertical flip is not: the sky never appears below the road in a forward-facing photo.
What DINOv2 captures	A self-supervised vision transformer trained to give the same representation to different views of the same image, so its embedding describes texture and shape rather than an object label; that suits "is this the same damage from another angle?".
Cosine similarity and threshold	Cosine of the angle between two L2-normalised vectors (1 = identical direction). The threshold trades precision for recall: higher means fewer false merges but more missed duplicates. Ours was tuned on 10 pairs and reported on 9 different pairs.
Domain shift and how to reduce it	The test data differs from training data (country, asphalt, camera, light), so performance drops (0.96 → 0.76 → 0.26). Reduce it with local labelled data, fine-tuning on a small local set, and augmentations that mimic local conditions.

Appendix F. Problems we hit and how we fixed them

Engineering problems found while building and testing the system, in the order they matter for the results. They are part of the decision log the brief asks for.

Problem	What we observed and why	Fix and effect
Photos analysed sideways	iPhone photos store the camera rotation in EXIF (orientation 6). The app read the raw pixels, so the model saw roads rotated by 90° and missed damage that was clearly visible.	Apply `ImageOps.exif_transpose` before detection in the app and in the batch map. Detections on phone photos recovered.
Default confidence 0.25 in an early app call	The app passed the Ultralytics default instead of the chosen threshold, so it showed many more false boxes than the evaluated model.	The threshold is read from `configs/model_config.json` by the detector class; a unit test checks the default equals the config.
Wrong checkpoint risk	Several similarly named files existed (published original, Kaggle "selected candidate", copies inside runs 24/25).	The adopted file is identified by its SHA-256 in the config and checked in tests and reports.
Duplicates without GPS	With the 0.80 threshold and no location, 12 of 24 different damages were merged on test pairs.	Separate, stricter 0.92 threshold chosen on tuning pairs only: 0 / 24 false merges on test (Section E5).
GPS lost when sharing photos	Photos sent through messaging apps lose their EXIF GPS, which the duplicate finder needs.	Original files were collected through Google Drive downloads; 59 of 60 duplicate-pair photos kept GPS.
Raw (DNG) photos	Some phone raw files could not be decoded by LibRaw.	Fallback to the largest JPEG preview embedded in the DNG; GPS read from EXIF separately.
HEIC photos	iPhone HEIC uploads could not be opened by Pillow.	Added `pillow-heif`; HEIC is accepted in the app.
Cloud app crash ("Oh no")	On Streamlit Community Cloud, 10 MB phone photos plus the YOLO and DINOv2 models exceeded the free memory limit; the process was killed without an error message.	Decode with JPEG draft mode and resize to at most 1,600 px before any processing. Memory stays within the limit; the detector still runs at 640 px so accuracy is unchanged.
Map tiles disappeared	The dark map provider started requiring an API key.	OpenStreetMap tiles with a CSS filter; no key or secret needed.
Arabic layout	Right-to-left text reversed mixed strings such as the product name and damage codes.	Left-to-right isolation for Latin fragments in the Arabic interface.
Notifications (scoped out)	We tried WhatsApp status notifications to citizens through the Twilio sandbox (template errors 21655 and authentication failures) and the Meta Cloud API (new business account locked, error 131031). Both require a verified paid business account.	Removed the feature from the app rather than ship something unreliable; citizens see status in "My reports". API keys were never committed; secrets stay in local `.env` files only.
Training failures	Run 4 stopped on GPU memory; run 16's standard recipe erased pretrained knowledge; run 26 (tiling) failed on a missing input path.	Smaller arms, the gentle low-learning-rate recipe (run 17), and run 26 reported as failed with no result claimed (Appendix B.2).